



[HOME](#)

[ABOUT](#)

[CONTENT](#)

QUẢN LÝ DỰ ÁN GIT

2026

Nhóm 1

1. **Nguyễn Hiếu**

2. **Hà Đức Kiên**

3. **Huỳnh Thảo Nhi**

4. **Nguyễn Chí Thành**



GIT

[HOME](#)

[ABOUT](#)

[CONTENT](#)

NỘI DUNG

01 Tổng quan về GIT

02 Nguyên lý hoạt động của GIT

03 Cơ bản về Git

04 Quản lý nhánh trong Git

05 WorkFlow

06 Làm việc với Remote Repository





[HOME](#)

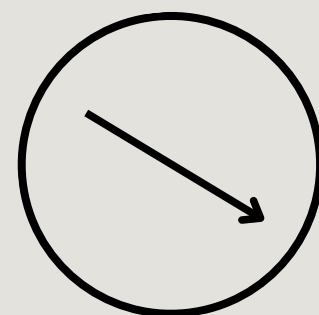
[ABOUT](#)

[CONTENT](#)

01

TỔNG QUAN VỀ GIT

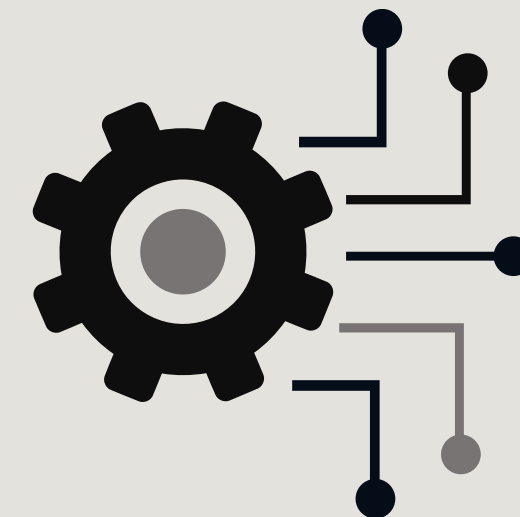
ĐẶT VẤN ĐỀ



1. Code gặp lỗi nhưng không có cách quay lại phiên bản trước đó?
2. Vô tình xóa mất một file quan trọng mà không có bản sao dự phòng?
3. Không có hệ thống quản lý phiên bản?



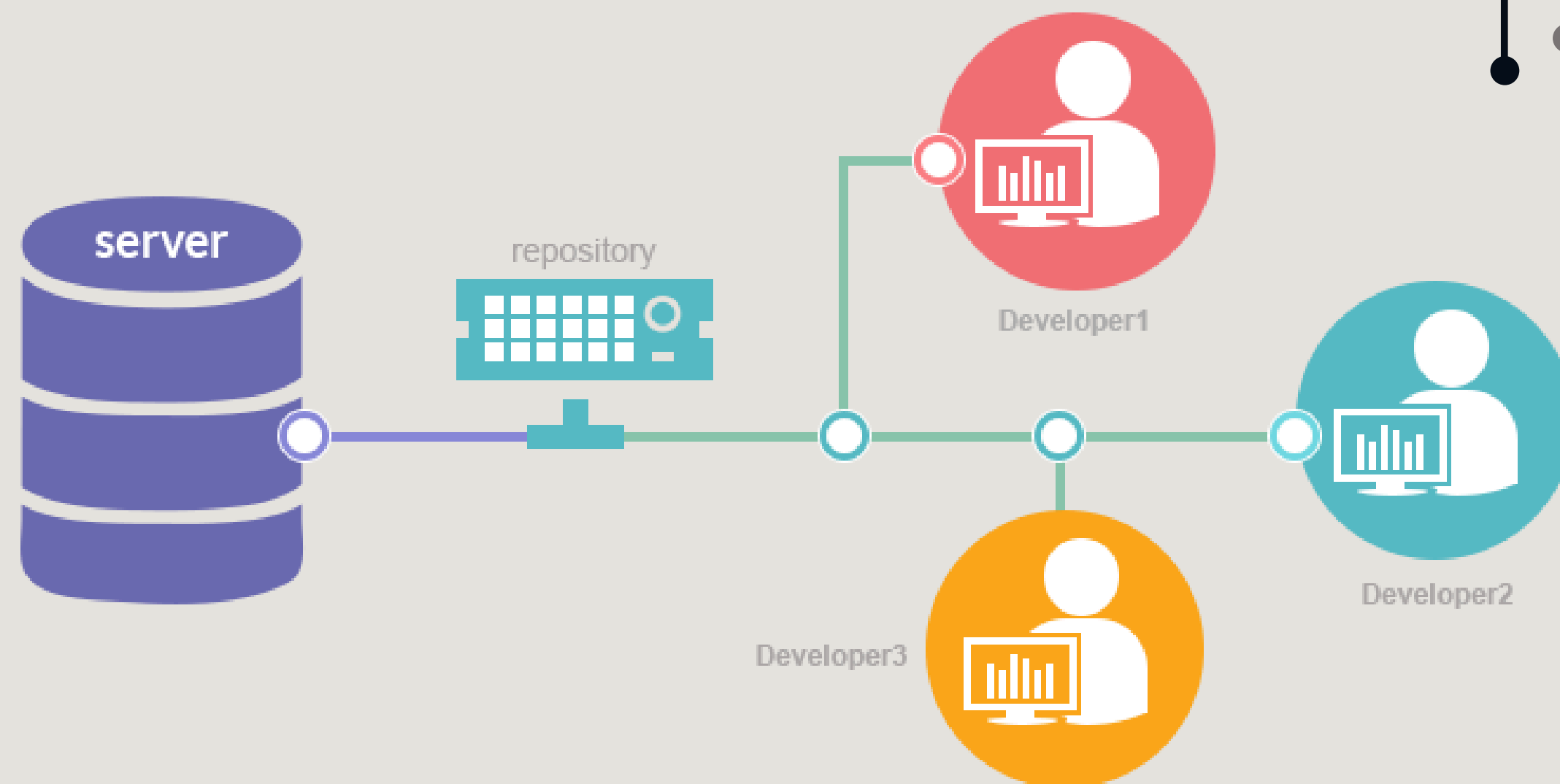
VCS - VERSION CONTROL SYSTEM



Version Control System (VCS) là hệ thống giúp **theo dõi và quản lý** các thay đổi của tệp tin theo thời gian.

Cho phép khôi phục phiên bản cũ, so sánh các thay đổi và làm việc nhóm hiệu quả.

Được sử dụng rộng rãi trong phát triển phần mềm để tránh mất dữ liệu và kiểm soát lịch sử chỉnh sửa.



VCS

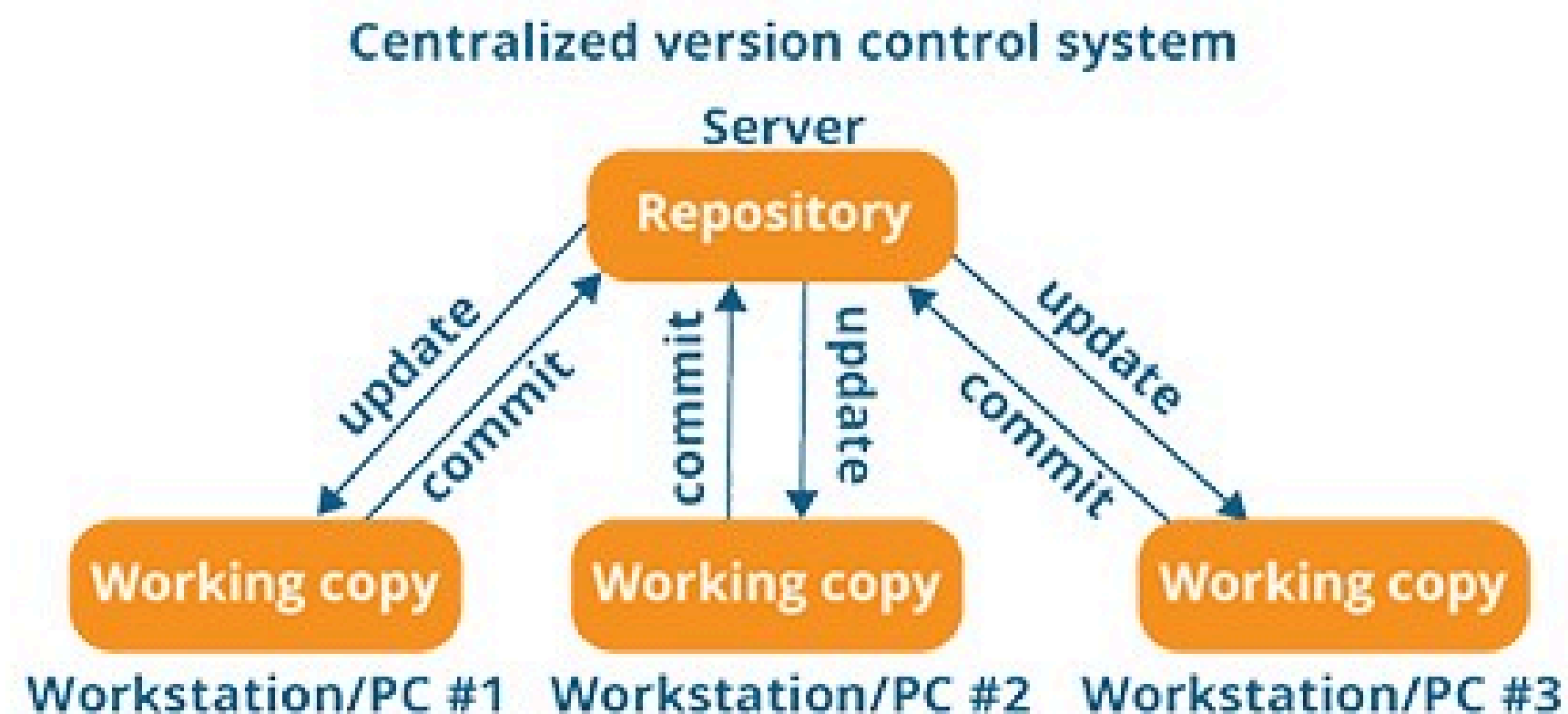
CVCS - Hệ thống tập trung

Một máy chủ trung tâm lưu trữ toàn bộ mã nguồn.

Các thành viên trong nhóm phải kết nối đến máy chủ để tải hoặc cập nhật mã nguồn.

Nếu máy chủ bị lỗi, toàn bộ dữ liệu có thể bị mất hoặc không thể truy cập.

Ví dụ: SVN, Perforce



VCS

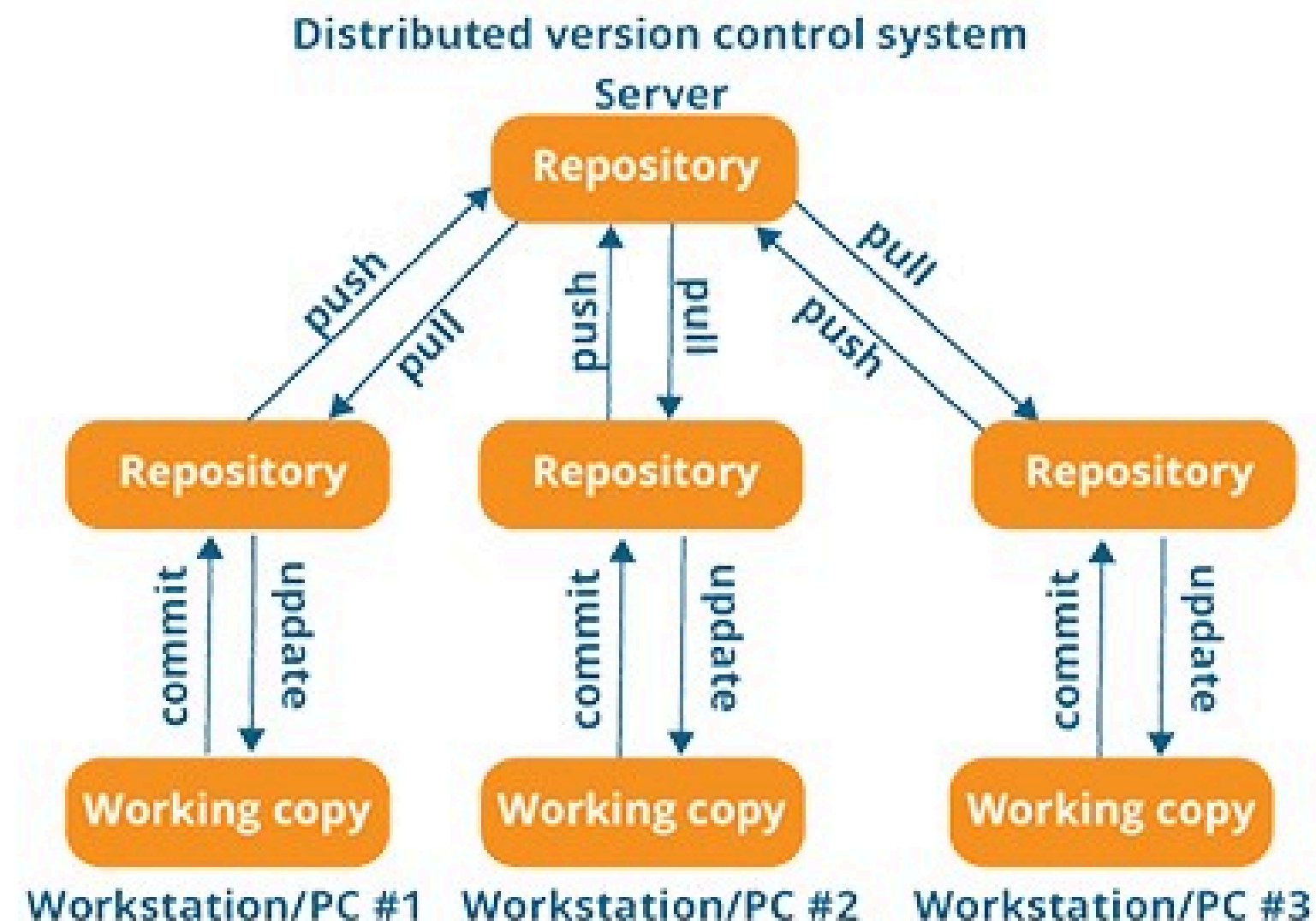
DVCS - Hệ thống phân tán

Mỗi thành viên trong nhóm có một bản sao đầy đủ của toàn bộ mã nguồn.

Không phụ thuộc vào máy chủ trung tâm, có thể làm việc ngoại tuyến.

Dễ dàng hợp nhất thay đổi từ nhiều thành viên khác nhau.

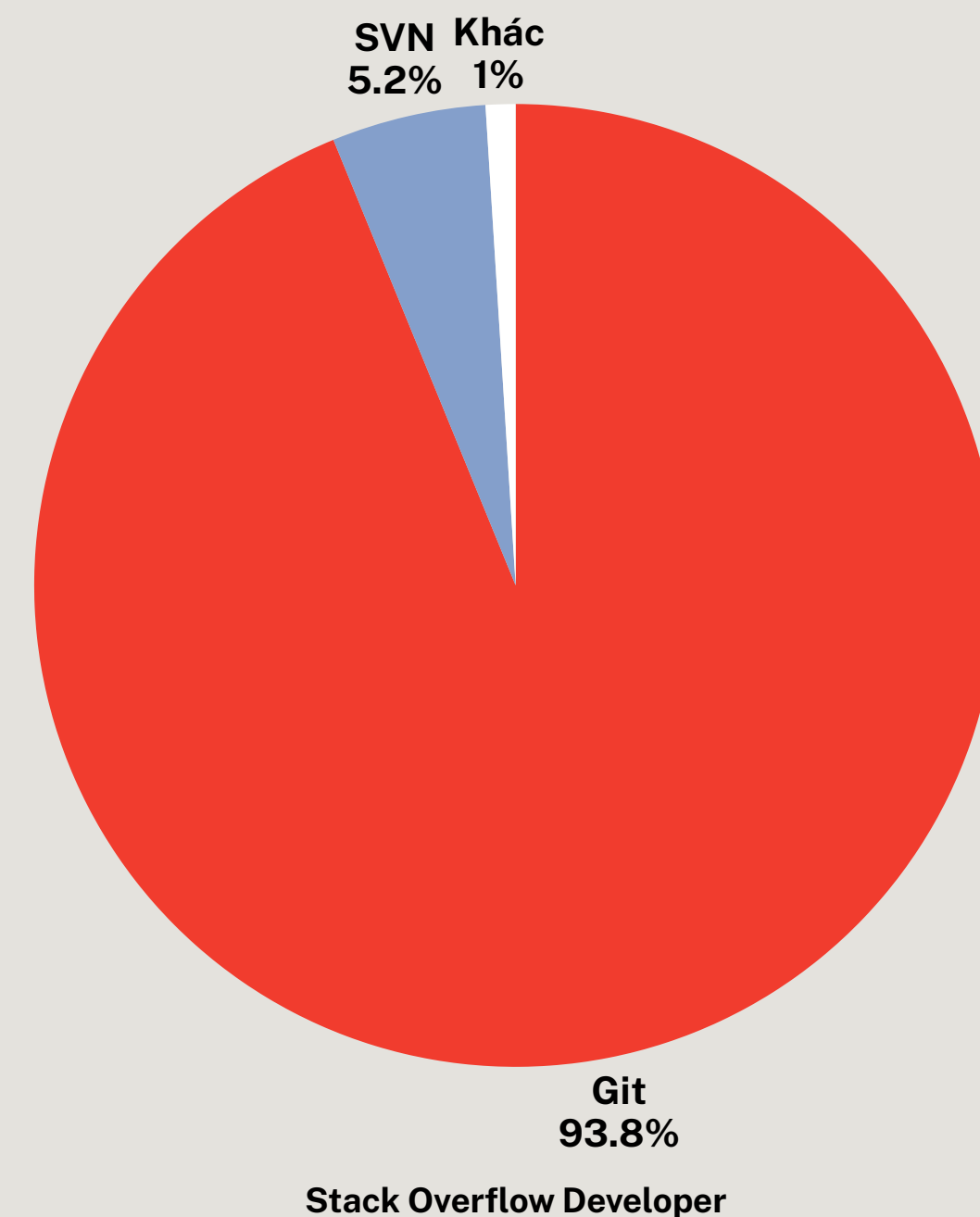
Ví dụ: Git, Mercurial



VCS



Biểu đồ tần suất sử dụng VCS năm 2024





Là một hệ thống quản lý phiên bản **phân tán** (Distributed Version Control System – DVCS) rất được ưa chuộng, được sử dụng để **theo dõi và quản lý** các sự thay đổi trong mã nguồn trong suốt quá trình phát triển phần mềm.

Tại sao nên sử dụng git

- 01 **Version Control:** Quản lý phiên bản hiệu quả.
- 02 **Collaboration:** Cho phép nhiều nhà phát triển cùng làm việc trên 1 dự án.
- 03 **Open Source:** Git là mã nguồn mở, được sử dụng rộng rãi, và được lưu trữ trên các nền tảng.
- 04 **Distributed (Phân tán):** Mỗi người có một bản sao đầy đủ của dự án, không phụ thuộc vào máy chủ trung tâm.





[HOME](#)

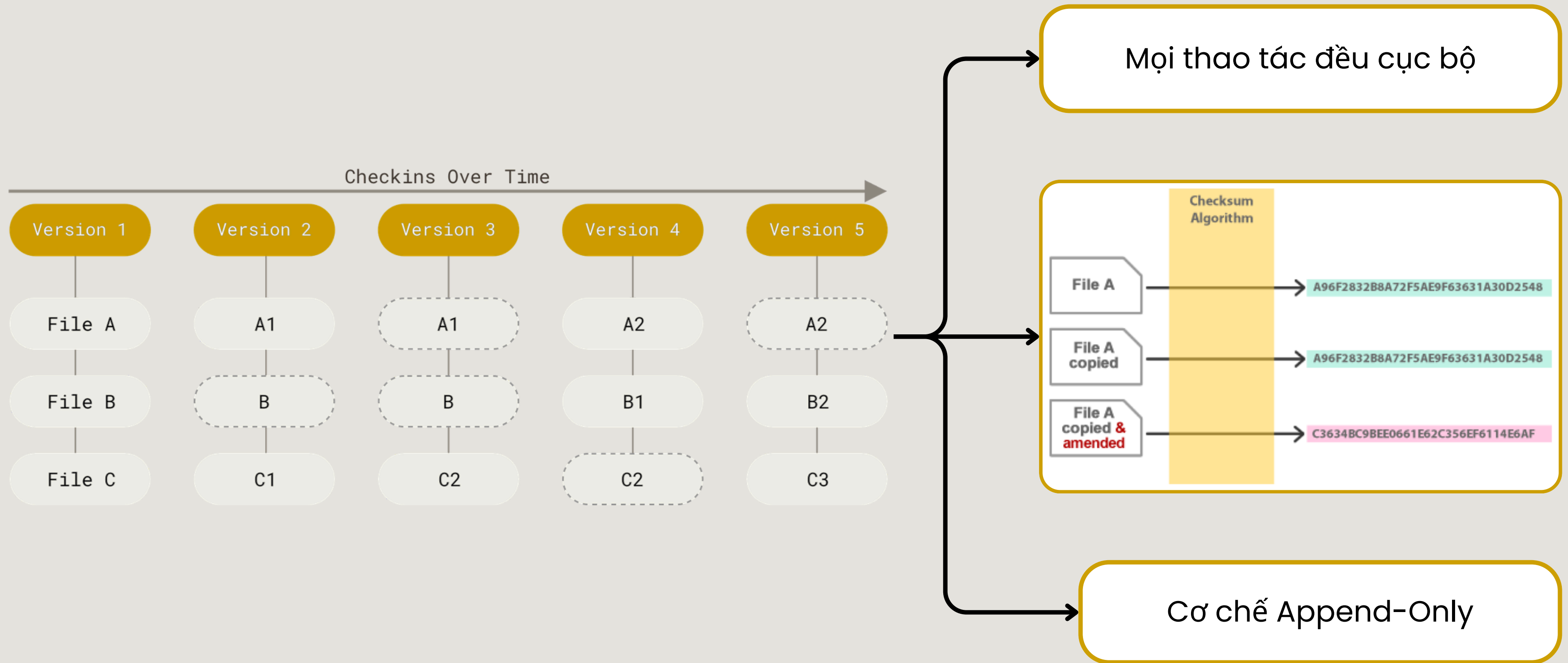
[ABOUT](#)

[CONTENT](#)

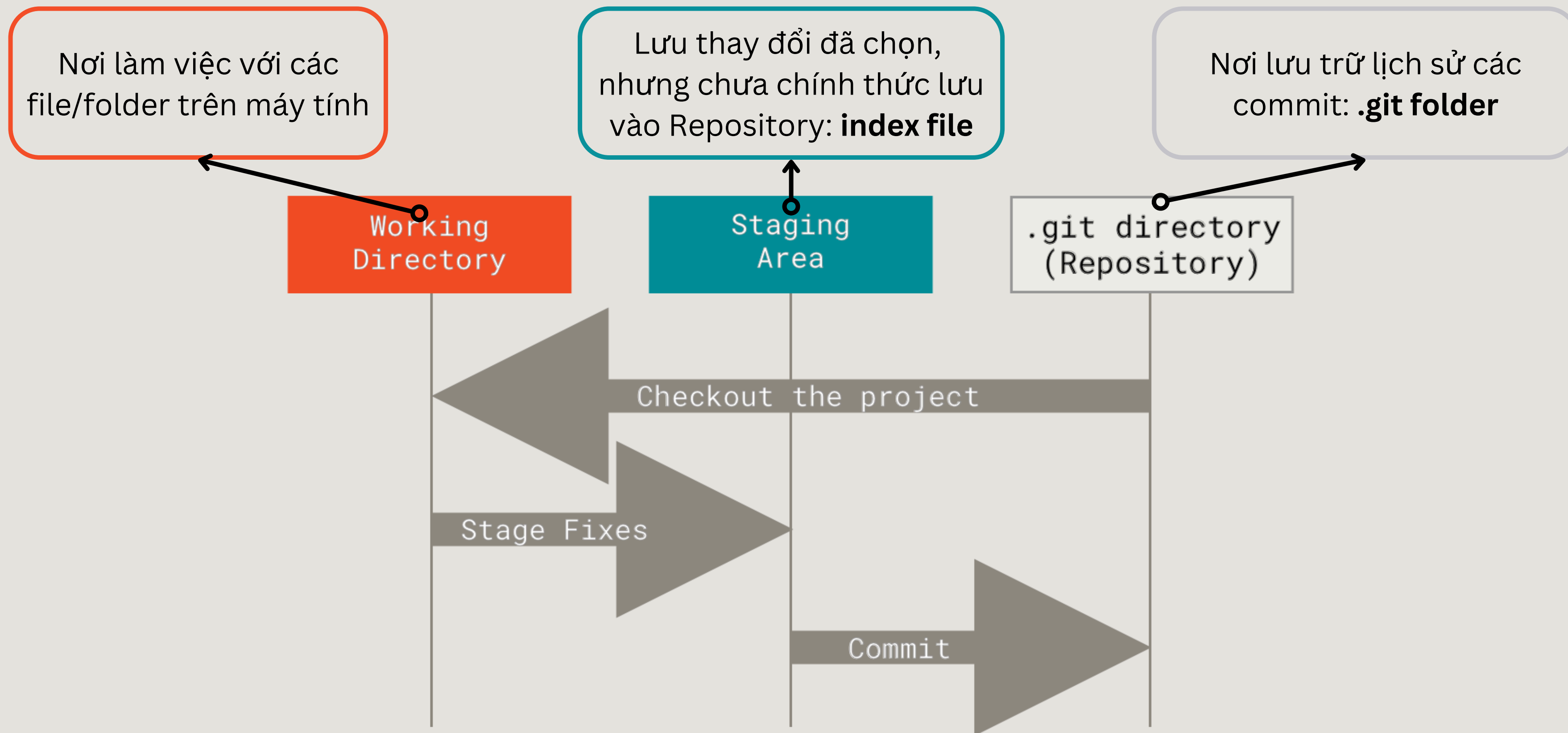
02

NGUYÊN LÝ HOẠT ĐỘNG CỦA GIT

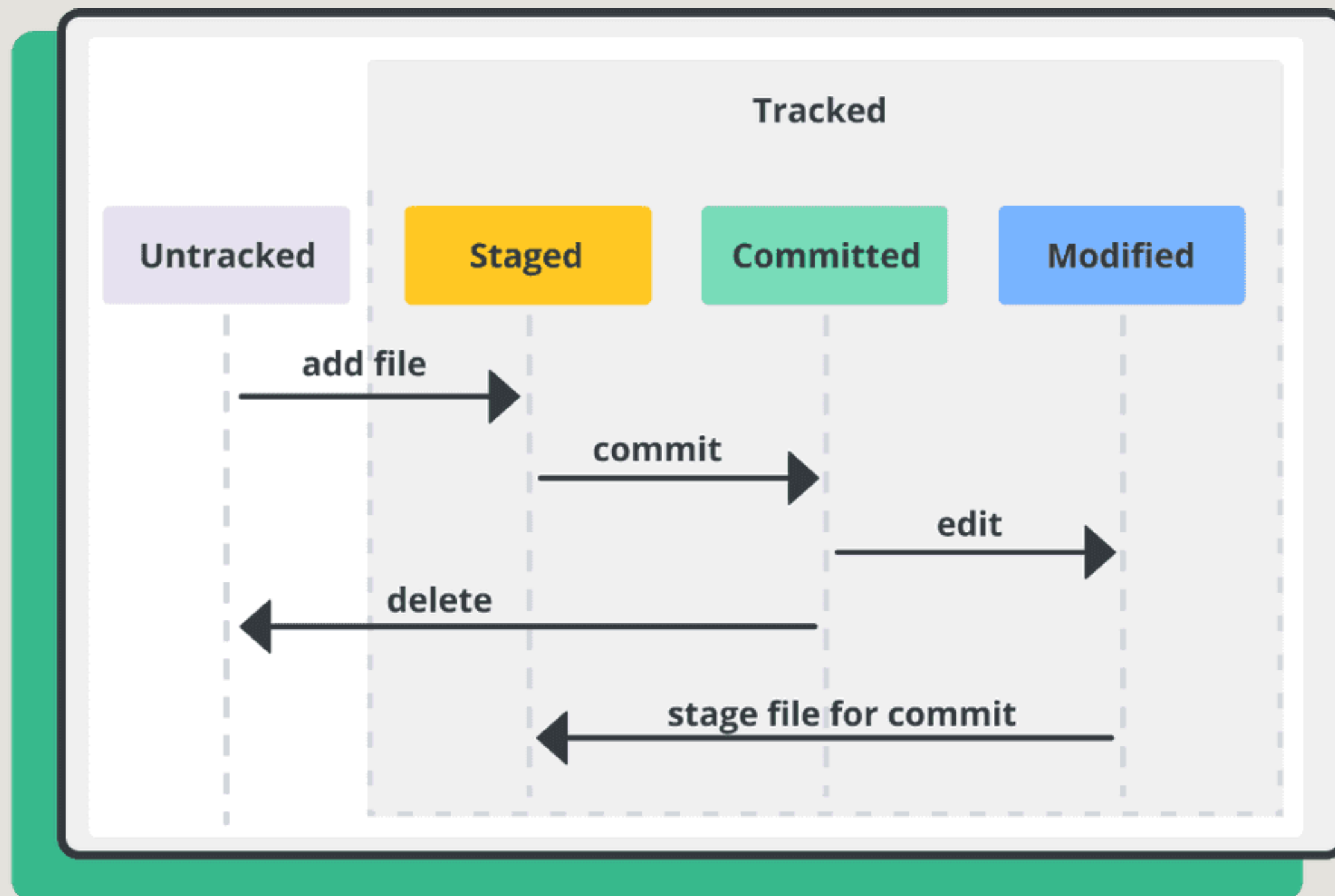
SNAPSHOT-BASED



3 VÙNG CỦA GIT



3 TRẠNG THÁI CỦA GIT



- **Untracked:** Có trong thư mục nhưng Git chưa từng add hoặc commit
- **Tracked:** Là file mà Git đã biết đến.
 - **Modified (Đã sửa đổi):** đã tác động vào file nhưng chưa lưu vào cơ sở dữ liệu.
 - **Staged (Đã chờ):** đánh dấu file/folder này để đưa vào snapshot tiếp theo.
 - **Committed (Đã lưu):** Dữ liệu đã nằm an toàn trong cơ sở dữ liệu cục bộ.



[HOME](#)

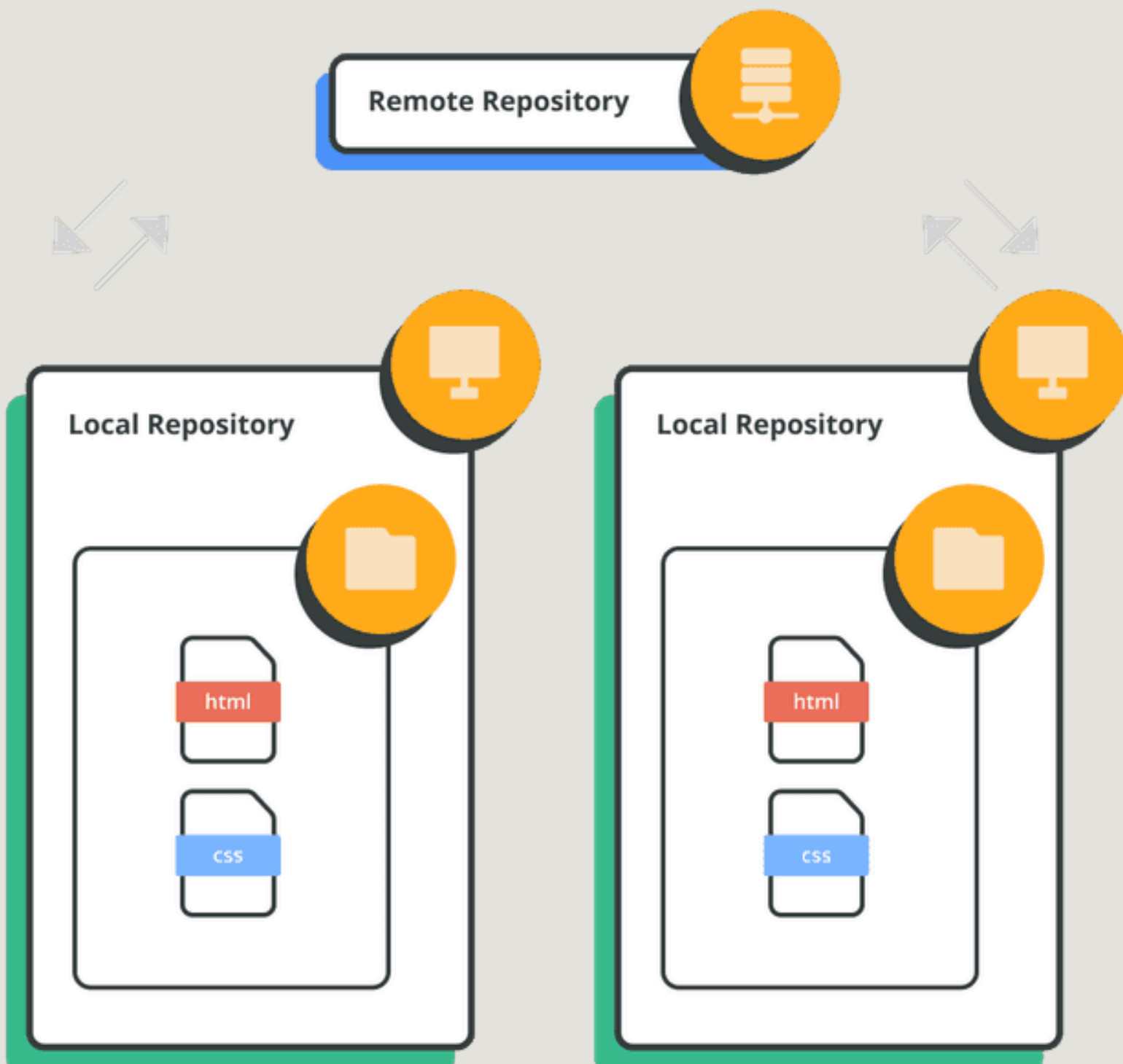
[ABOUT](#)

[CONTENT](#)

03

CƠ BẢN VỀ GIT

REPOSITORY



Remote Repository

- Là một **kho lưu trữ đặt trên server** chuyên dụng giúp tạo nền tảng chung cho việc lưu trữ và quản lý mã nguồn dự án
- Ví dụ: GitHub, GitLab, Bitbucket,...

Local Repository

- Là một **thư mục trên máy tính** chứa toàn bộ mã nguồn của dự án, lịch sử thay đổi (commit history) và các nhánh (branches)
- Đồng bộ hóa với remote repository bằng các lệnh của git

REPOSITORY

	Local Repository	Remote Repository
Vị trí lưu trữ	Trên ổ cứng của người dùng.	Trên máy chủ hoặc nền tảng đám mây.
Truy cập	Chỉ có thể truy cập trên máy cục bộ.	Có thể truy cập từ nhiều máy khác nhau.
Mục đích	Lưu trữ thay đổi cục bộ, giúp theo dõi mã nguồn.	Lưu trữ và chia sẻ code với nhóm, hỗ trợ làm việc nhóm.
Làm việc nhóm	Chỉ một người làm việc trên một máy.	Hỗ trợ nhiều người cùng làm việc từ nhiều thiết bị khác nhau.
Cần kết nối mạng?	Không cần kết nối internet.	Cần internet để đồng bộ dữ liệu.

CÁC CÂU LỆNH CƠ BẢN

GIT INIT

Khởi tạo local repository – thư mục .git ẩn để Git quản lý phiên bản của dự án.

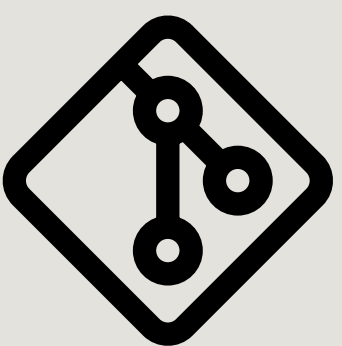
Lệnh: **git init**



git init



CÁC CÂU LỆNH CƠ BẢN

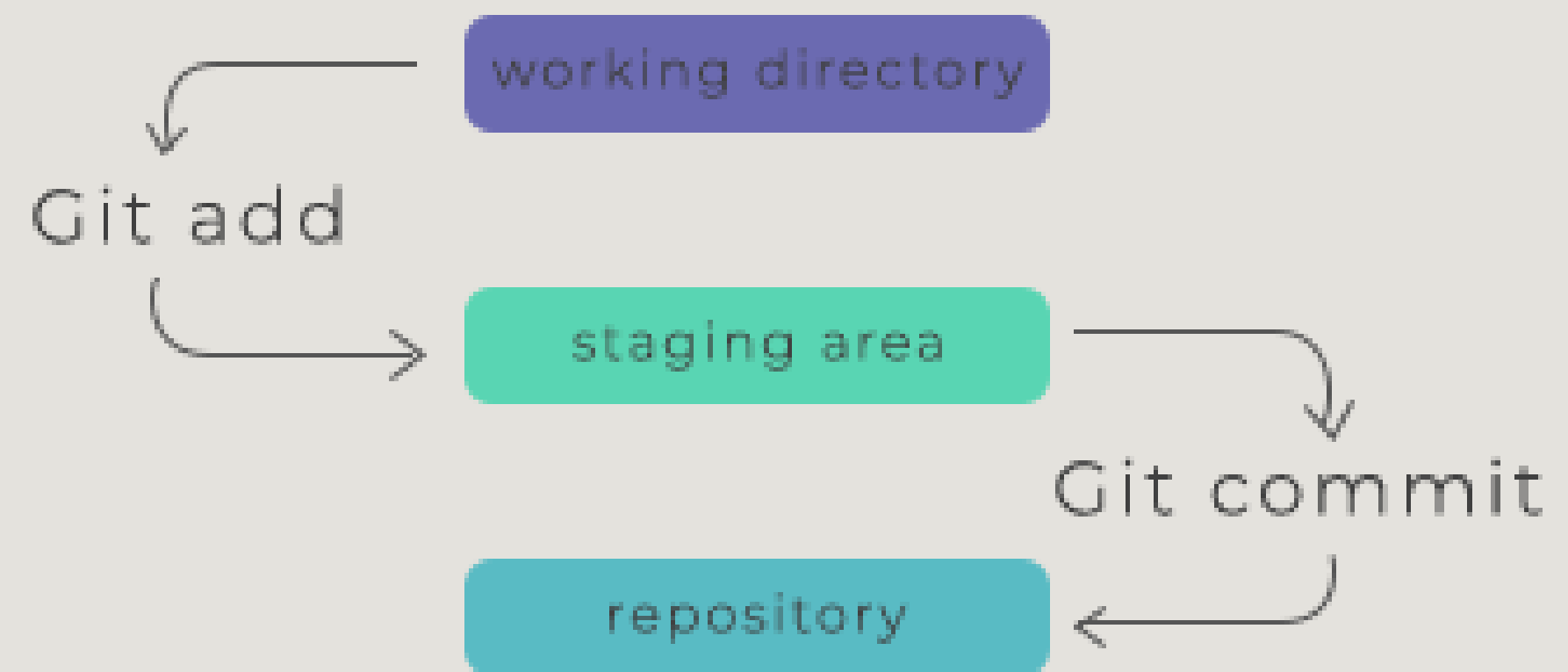


GIT ADD

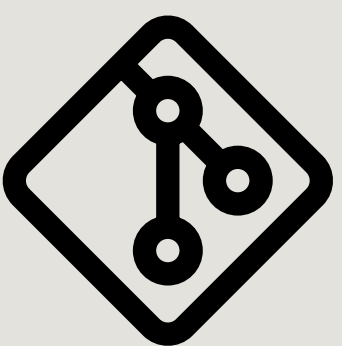
Thêm thay đổi của file ở Working Directory vào Staging Area

Lệnh:

- **git add .**
- **git add <tên_file>**



CÁC CÂU LỆNH CƠ BẢN

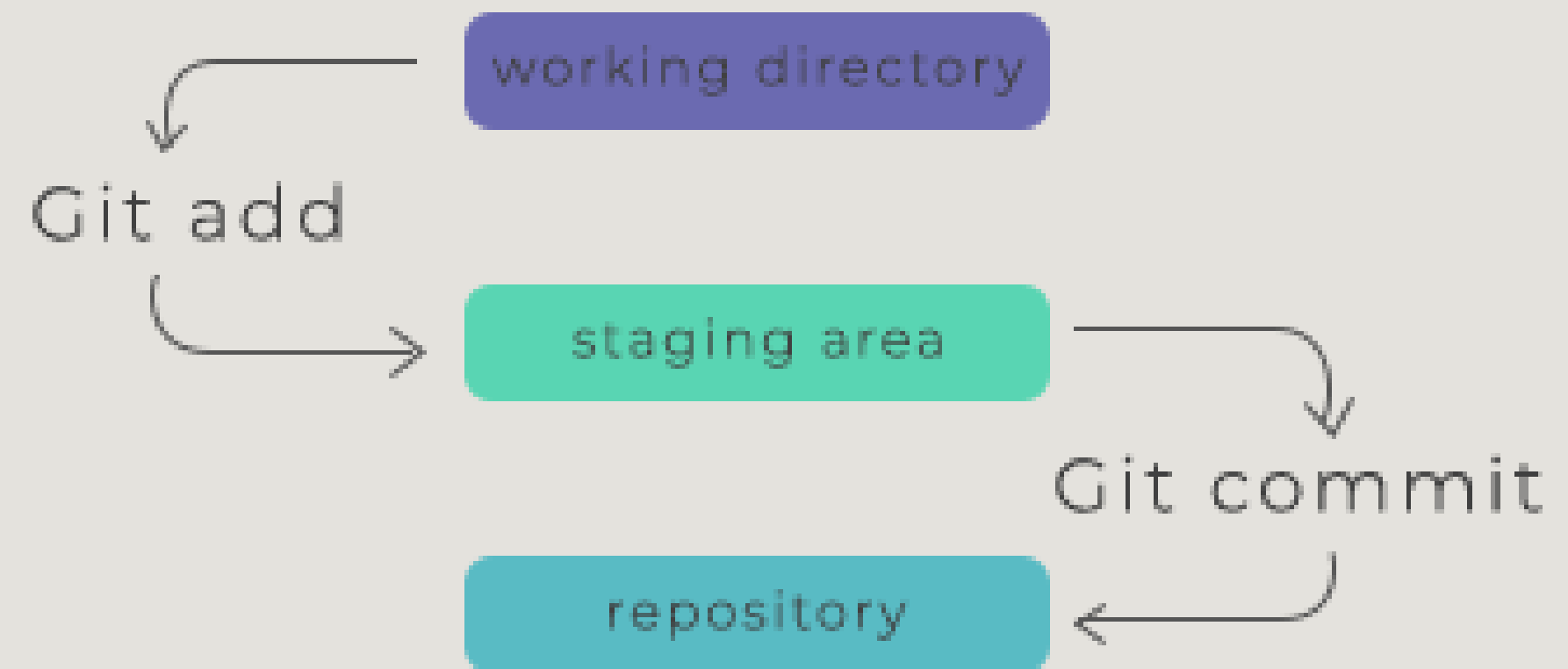


GIT COMMIT

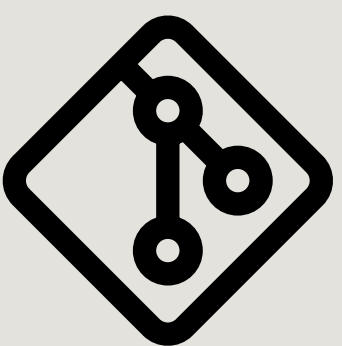
Lưu một bản ghi của tất cả các file ở Staging Area

Lệnh:

- **git commit -m "tin_nhắn"**
- **git commit -a**



CÁC CÂU LỆNH CƠ BẢN



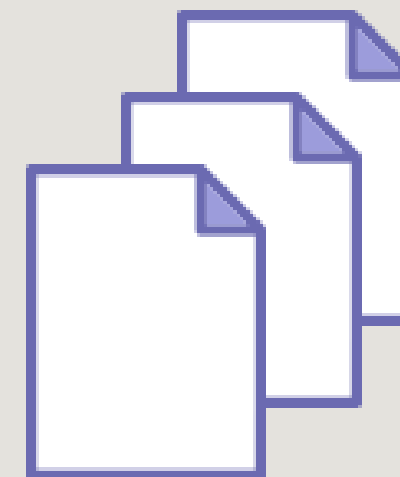
GIT STATUS

Xác định trạng thái file ở Working Directory và Staging Area

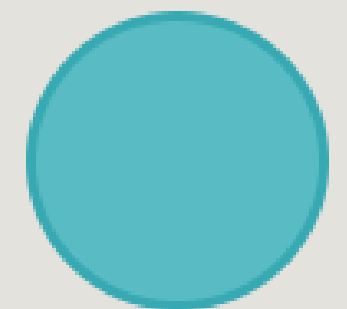
Lệnh:

- **git status**
- **git status -s**

Git Status

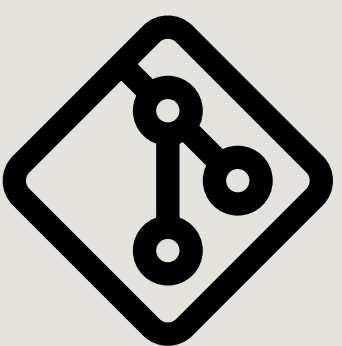


Working Directory



Stage Snapshot

CÁC CÂU LỆNH CƠ BẢN



GIT LOG

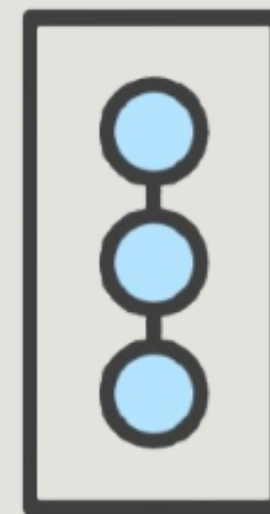
Dùng để xem lịch sử commit trong Git.

Lệnh:

- **git log**
- **git log -p**
- **git log --author="<Tên người dùng>"**

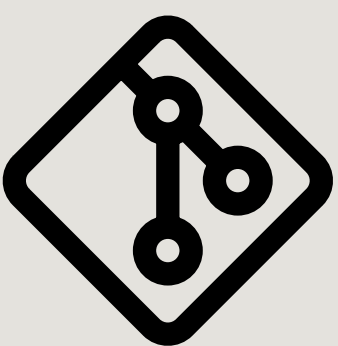
A light blue rectangular box with a black border containing the text "Git Log".

Git Log



Committed
History

CÁC CÂU LỆNH CƠ BẢN



GIT DIFF

Xem chi tiết thay đổi giữa Working Directory
và Staging Area

Lệnh:

- **git diff**
- **git diff --staged**





[HOME](#)

[ABOUT](#)

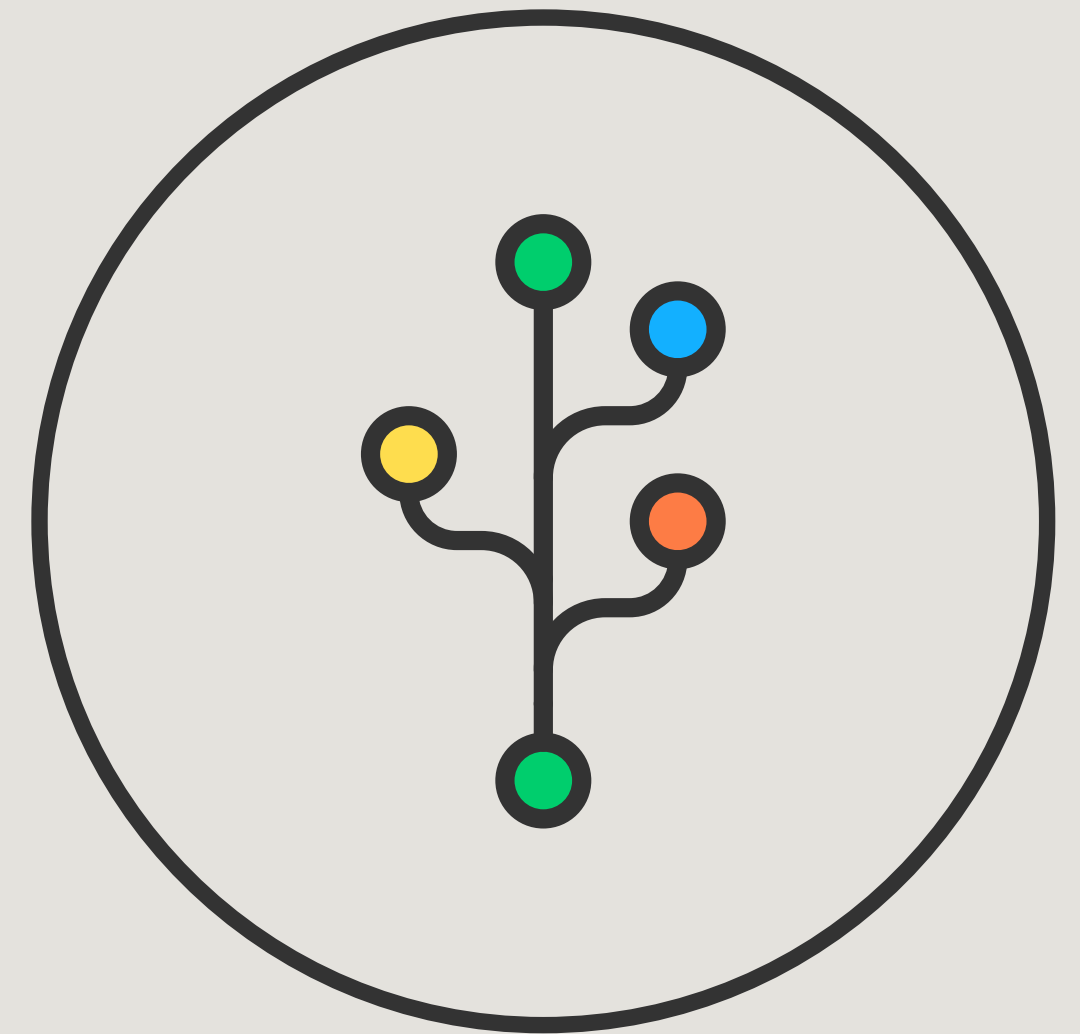
[CONTENT](#)

04

QUẢN LÝ NHÁNH TRONG GIT

BRANCH

- **Branch (nhánh)** là một con trỏ di động, trỏ đến một commit trong repository
- Gồm:
 - commit của branch cha
 - commit của branch hiện tại
- Vai trò:
 - Phát triển tính năng độc lập
 - Tạo bản vá lỗi nhanh chóng
 - Tăng khả năng cộng tác





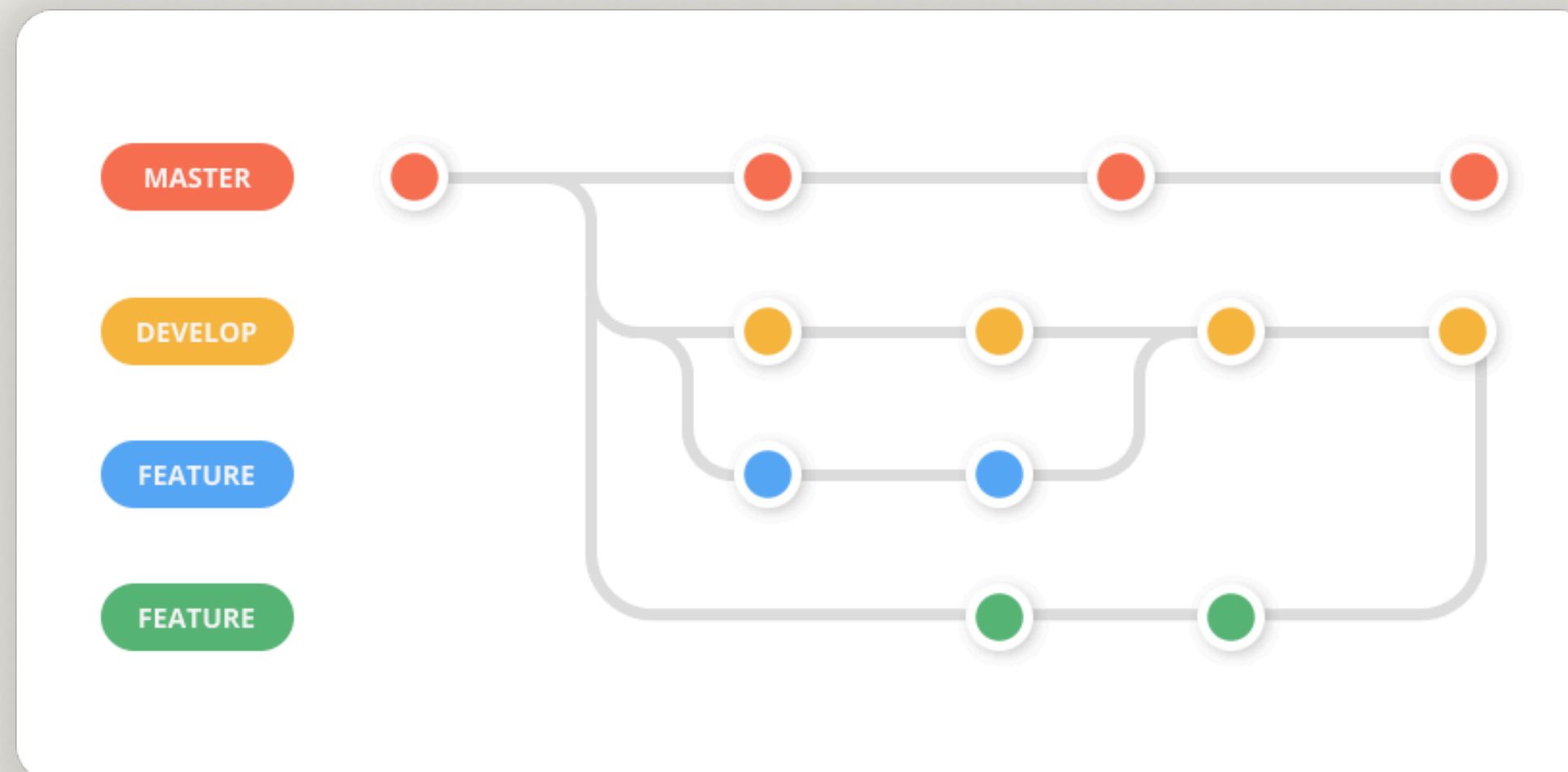
PHÂN LOẠI BRANCH

1. Theo chức năng:

- Develop
- Feature
- Main
- Hotfix
- ...

2. Theo vòng đời

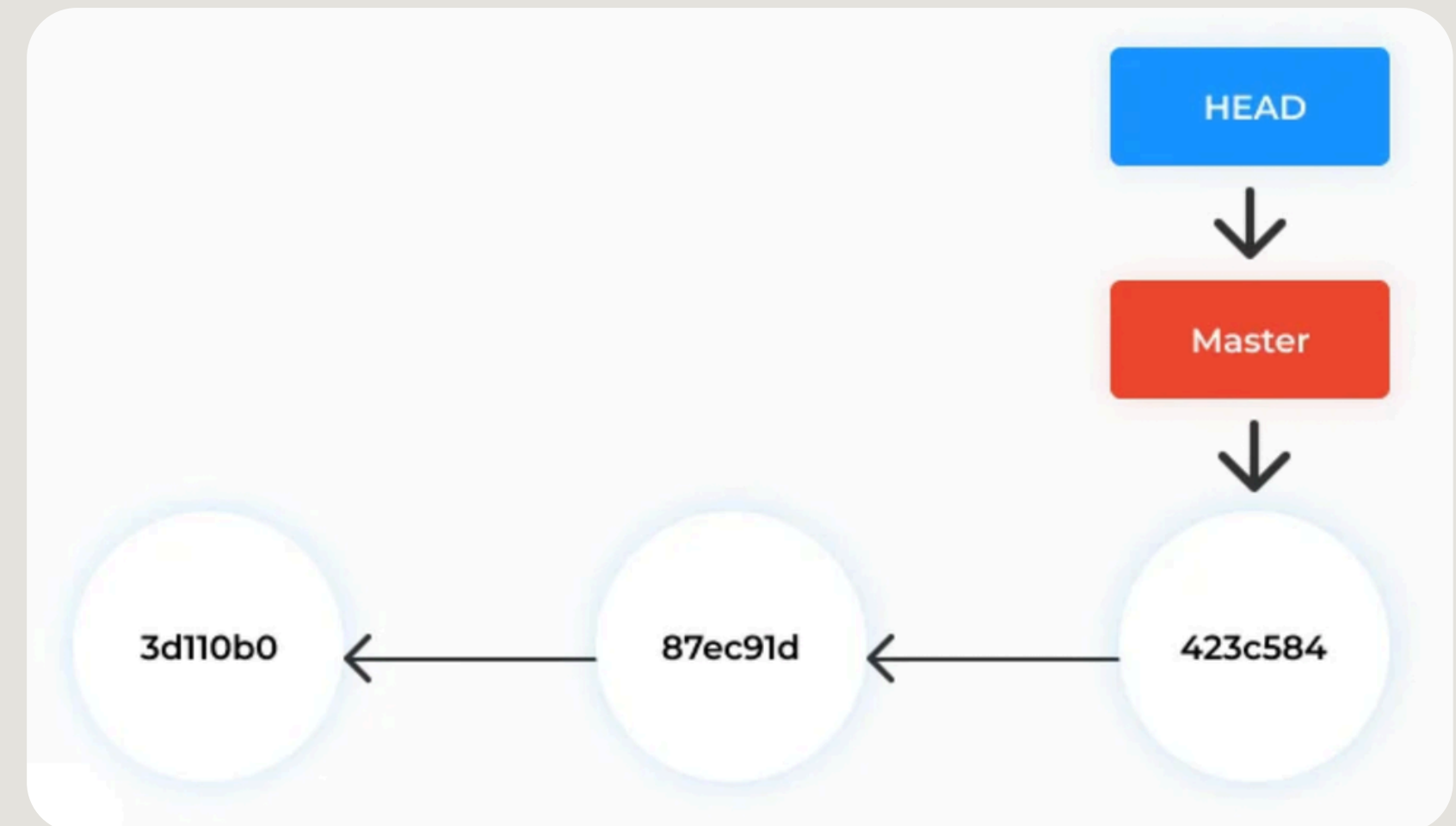
- Long-Running Branches
- Short-Lived Branches





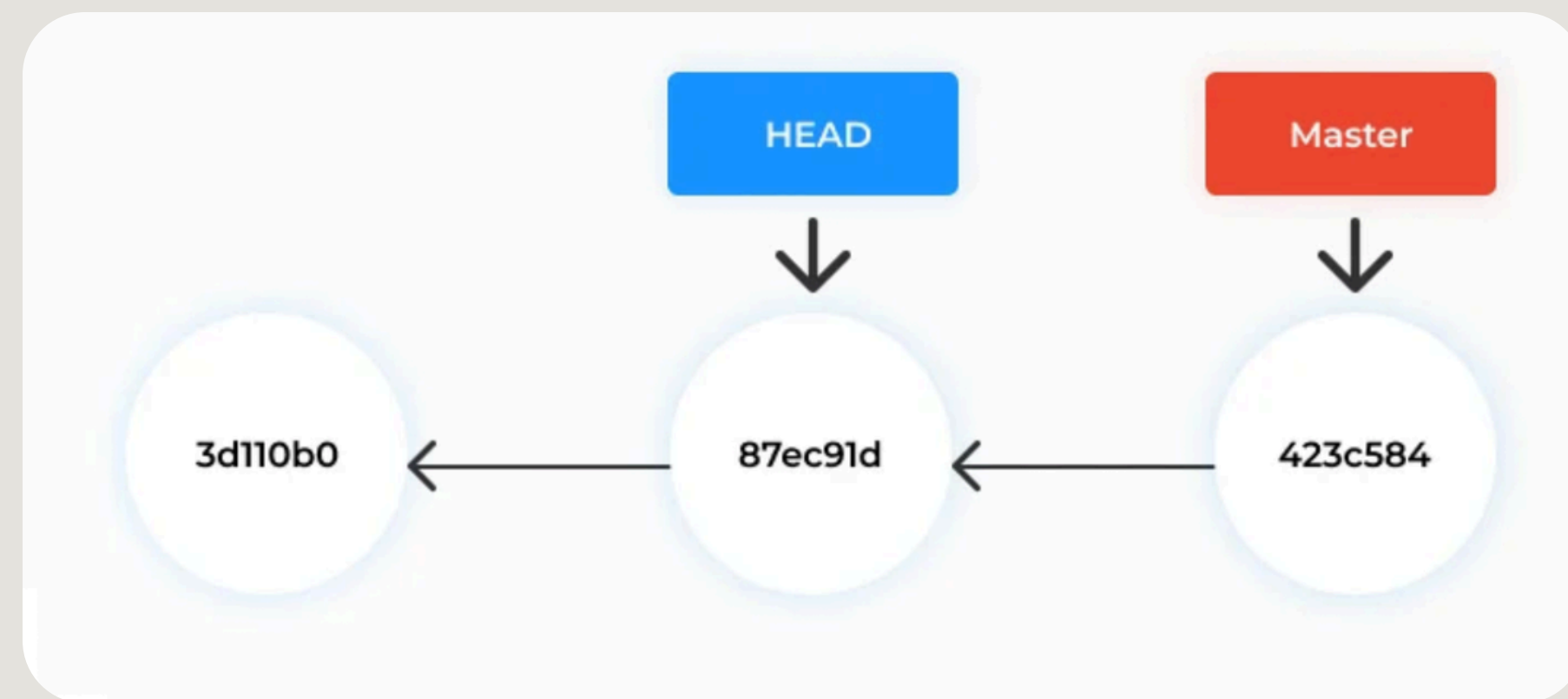
HEAD

- HEAD là một con trỏ, **trỏ đến nhánh hiện tại** đang hoạt động.
- Vai trò:
 - Xác định vị trí hiện tại
 - Cập nhật khi chuyển nhánh



DETACHED HEAD

- Detached HEAD là trạng thái con trỏ HEAD **trỏ đến một commit cụ thể**
 - Đặc điểm: Các commit mới không thuộc về nhánh nào
 - Mục đích: Sử dụng tạm thời
- **Chú ý:** Khi ở trạng thái detached HEAD muốn lưu các thay đổi đó, hãy tạo một nhánh mới.



CÁC CÂU LỆNH CƠ BẢN

GIT BRANCH

Xem nhánh

Lệnh:

- **git branch**
- **git branch -v**

Tạo nhánh mới

Lệnh:

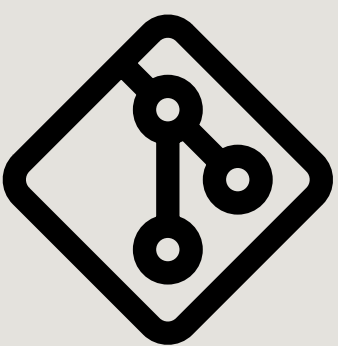
- **git branch <tên_nhánh_mới>**

Xóa nhánh

Lệnh:

- **git branch -d <tên_nhánh>**
- **git branch -D <tên_nhánh>**

CÁC CÂU LỆNH CƠ BẢN



GIT CHECKOUT

Di chuyển HEAD trở vào nhánh

Lệnh:

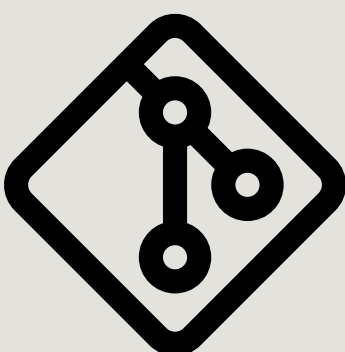
- **git checkout <tên_nhánh>**
- **git checkout -b <tên_nhánh_mới>**

```
$ git checkout master
```

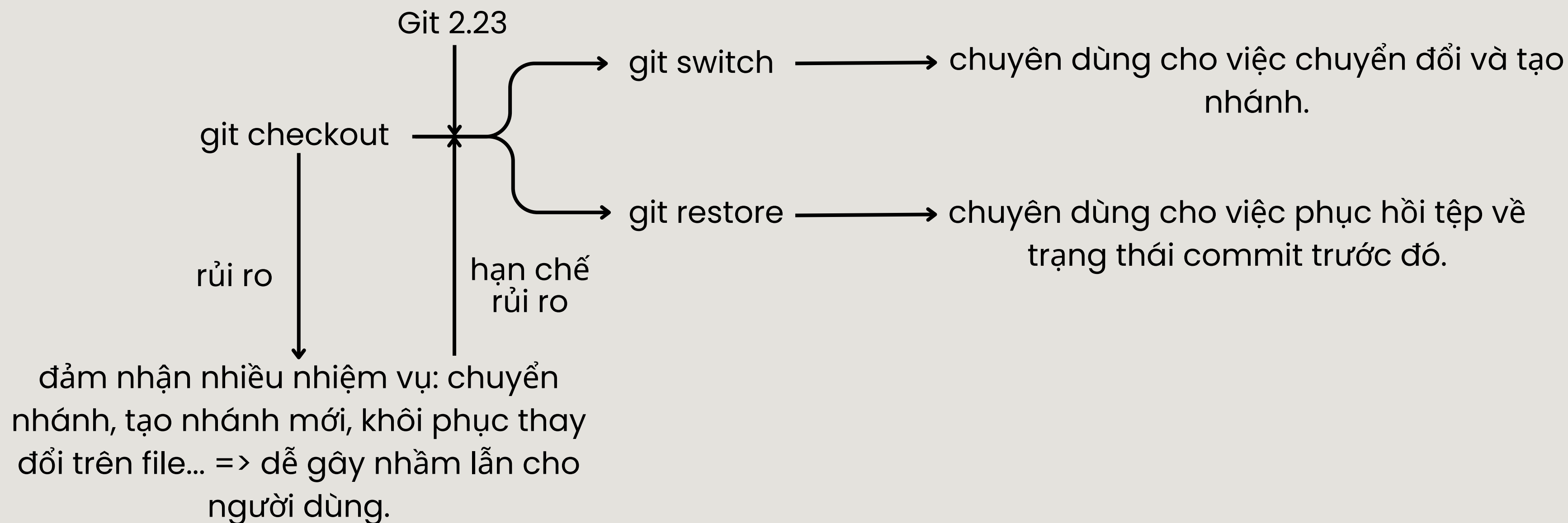


```
$ git checkout a05ef02
```

CÁC CÂU LỆNH CƠ BẢN

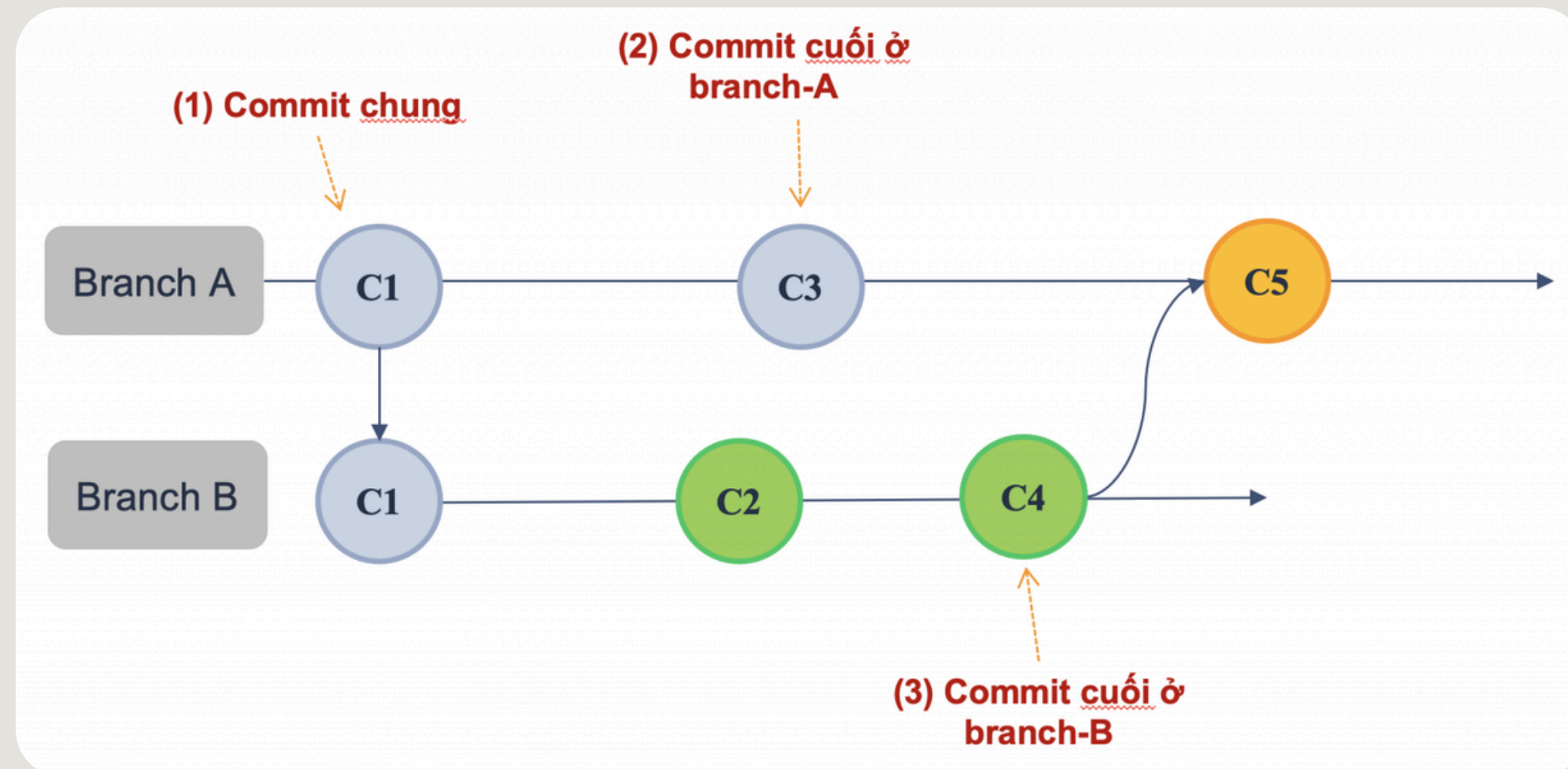
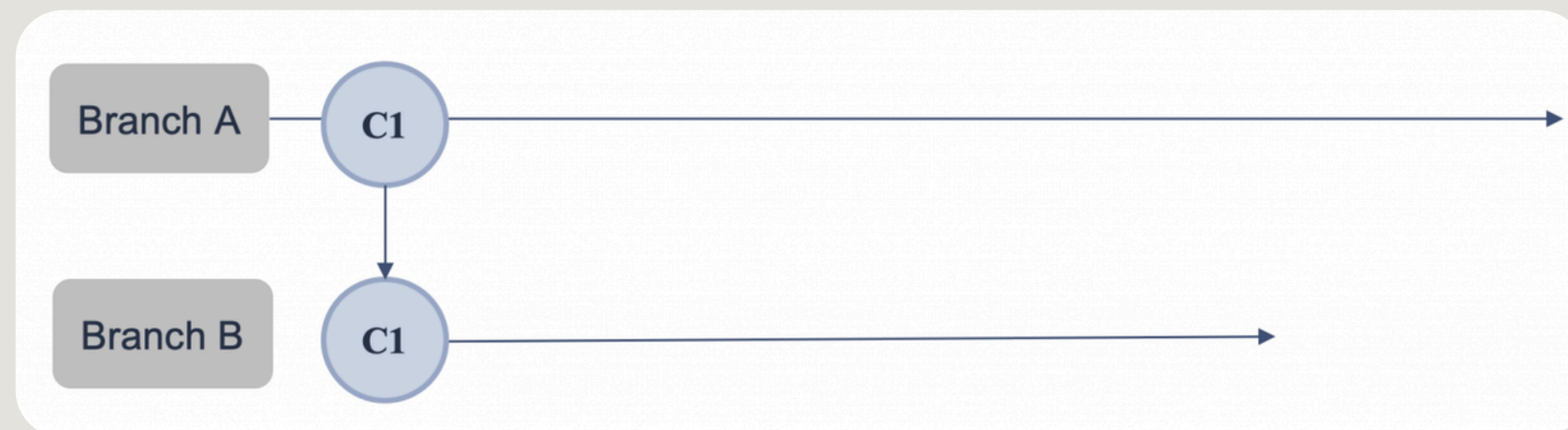


GIT SWITCH / GIT RESTORE



MERGE

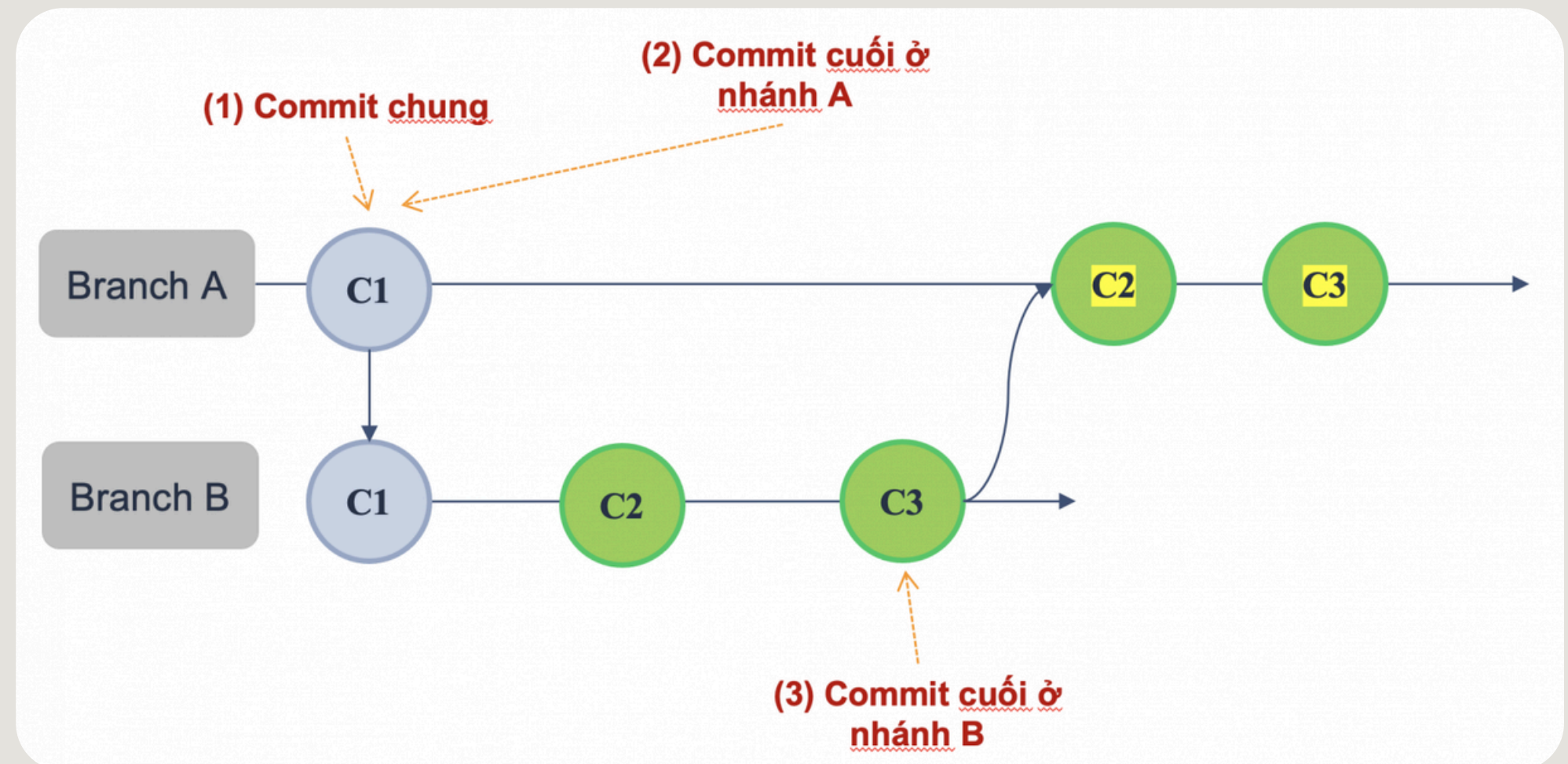
- Gộp nhánh khác vào nhánh hiện tại
- Điều kiện: hai nhánh có ít nhất 1 commit chung
- Lệnh: **git merge <tên_nhánh>**
- Lưu ý: Tránh merge từ master/main vào nhánh khác



PHÂN LOẠI MERGE

Fast-forward merge

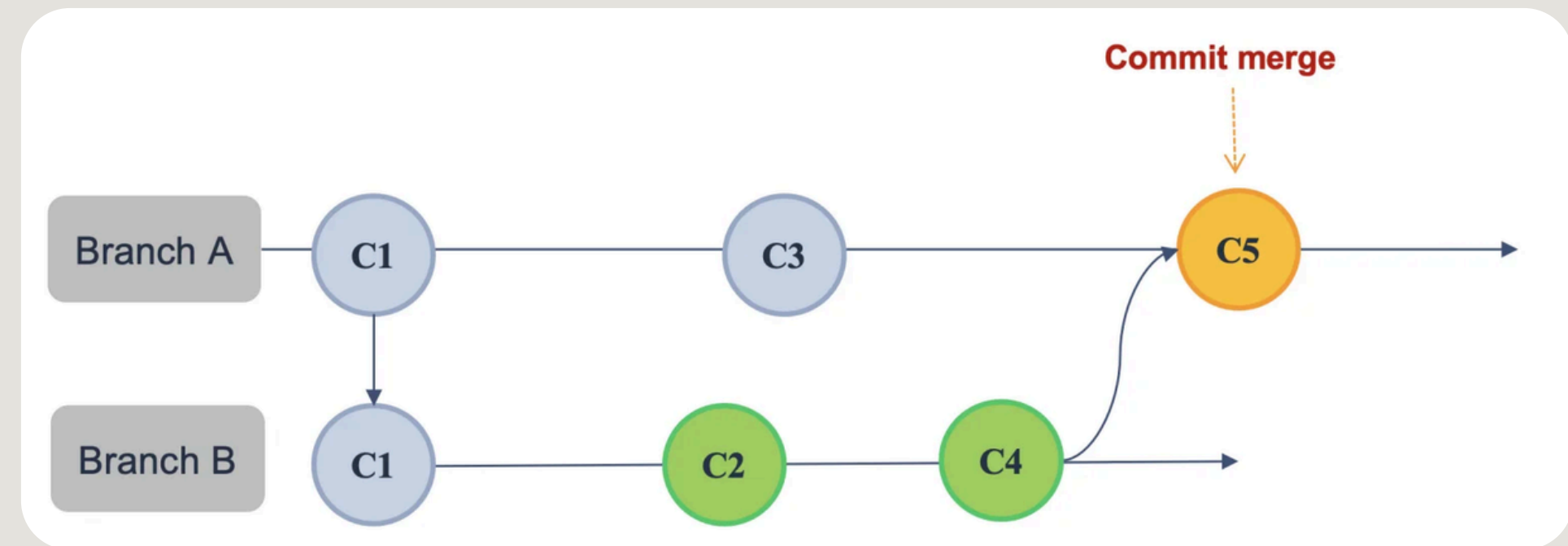
- Nhánh hiện tại không có thêm thay đổi so với nhánh được tách
- Đặc điểm:
 - Không tạo ra commit merge
 - Lịch sử commit được tuyến tính



PHÂN LOẠI MERGE

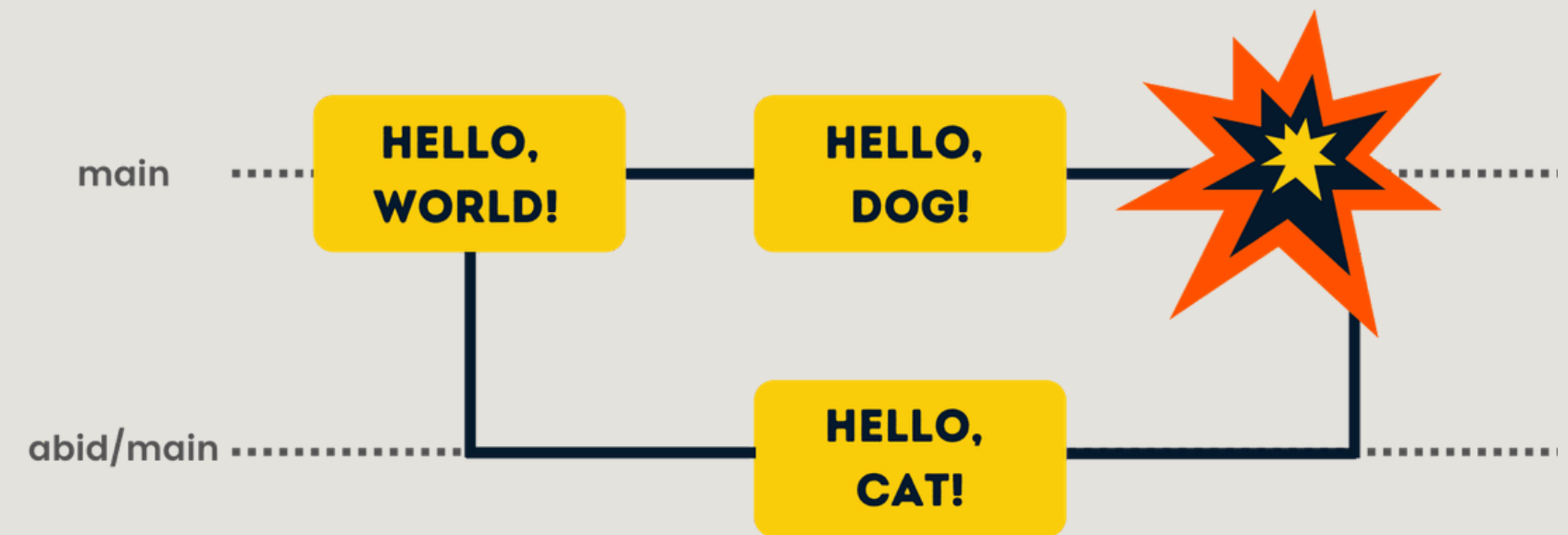
Three-way merge

- Nhánh hiện tại có thêm thay đổi so với nhánh được tách
- Đặc điểm:
 - Tạo ra một commit merge
 - Lịch sử commit không tuyến tính
 - Có thể xảy ra conflict nếu cùng tập tin



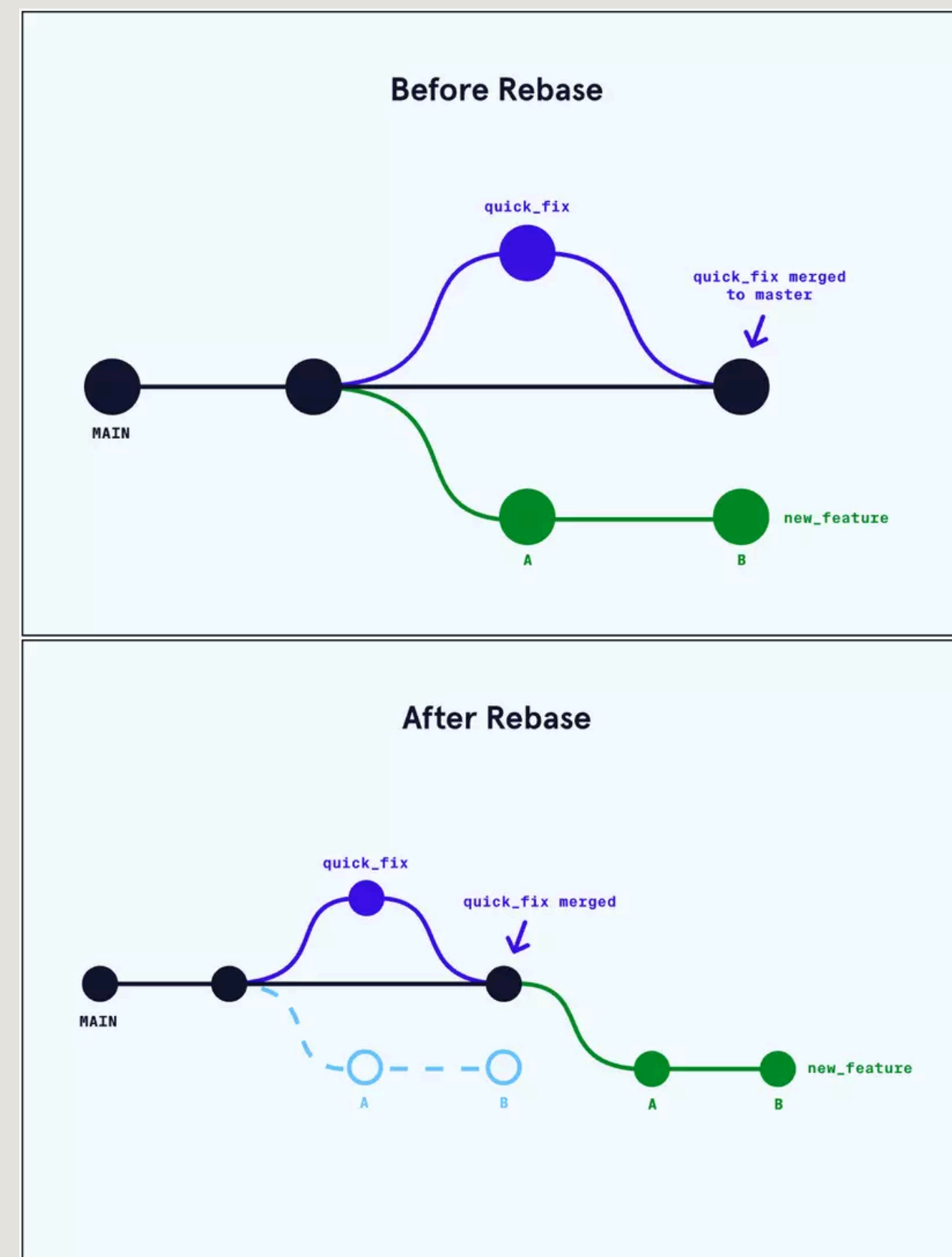
MERGE CONFLICT

- Git không tự động hợp nhất 2 nhánh do có nội dung xung đột trong file
- Khi 2 nhánh cùng chỉnh sửa một dòng code hoặc file.
- Xử lý Merge conflict:
 - B1: Kiểm tra file conflict
 - B2: Quyết định: giữ code nào/ viết code mới để hòa hợp cả 2
 - B3: Đánh dấu đã giải quyết: `git add <tên_file>`
 - B4: Tạo commit mới



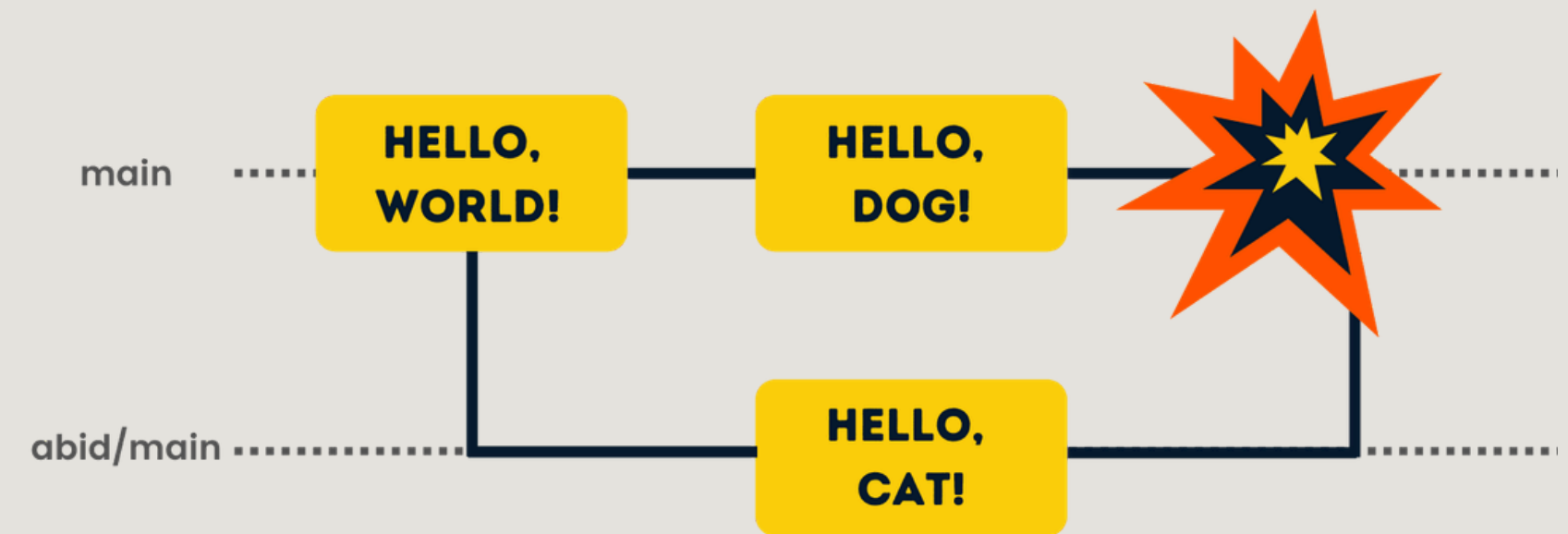
REBASE

- Gộp nhánh khác vào nhánh hiện tại bằng cách đặt lại commit lên trên
- Điều kiện: hai nhánh có ít nhất 1 commit chung
- Lệnh: **git rebase <tên_nhánh>**
- Lưu ý: Rebase sẽ viết lại lịch sử (tạo commit mới)



MERGE CONFLICT WITH REBASE

- Git không thể tiếp tục rebase khi không “áp lại commit” do có xung đột trong code
- Cùng một đoạn code bị thay đổi ở cả nhánh hiện tại và nhánh gốc → không biết chọn phiên bản nào
- Xử lý merge conflict khi dùng rebase:
 - B1: Git dừng tại commit bị conflict → Kiểm tra file conflict
 - B2: Quyết định: giữ code nào/ viết code mới để hòa hợp cả 2
 - B3: Đánh dấu đã giải quyết: `git add <tên_file>`
 - B4: Tiếp tục quá trình rebase: `git rebase --continue`





[HOME](#)

[ABOUT](#)

[CONTENT](#)

05

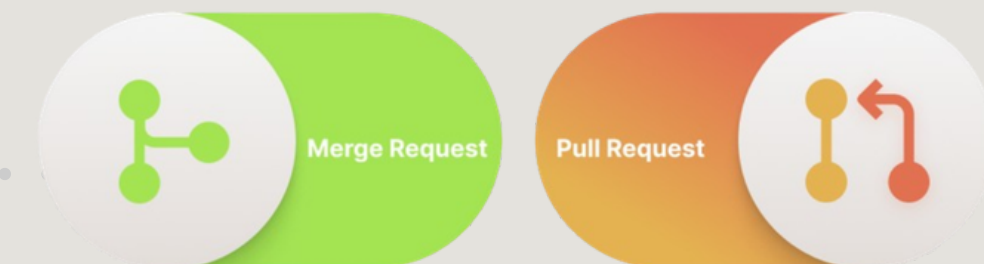
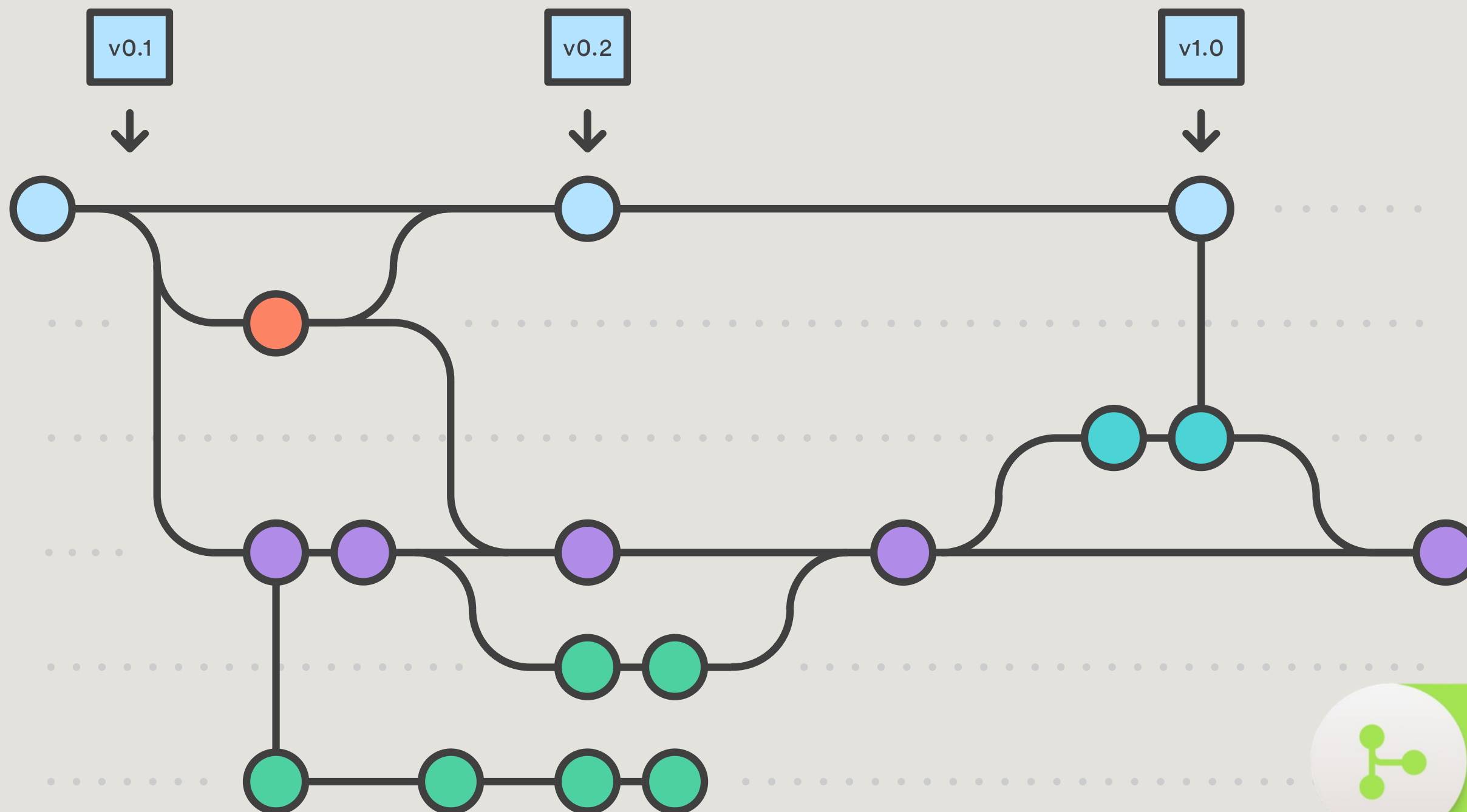
GIT WORKFLOW



Git Workflow là quy trình làm việc với Git để quản lý mã nguồn hiệu quả trong dự án.

Xác định cách thức setup các loại branch, cách merge và cộng tác giữa các thành viên.

05 - Git Workflow



MỘT SỐ NGUYÊN TẮC KHÁC

- Branch:
 - Không nên làm việc trực tiếp trên main/master
 - Quy ước đặt tên: chức_năng/
 - Ví dụ: feature/login, ...
- Commit:
 - Commit thường xuyên, có ý nghĩa
 - Format phổ biến: <type>: <mô tả ngắn> <mô tả chi tiết nếu cần>
 - Ví dụ: fix: correct login validation logic
 - Sử dụng `.gitignore`

Git Ignore File

Specifying files to ignore with `.gitignore`

```
# Common files to ignore
*.log
node_modules/
.DS_Store
.env
build/
dist/

# But keep this file
!important.log
```



[HOME](#)

[ABOUT](#)

[CONTENT](#)

06

LÀM VIỆC VỚI REMOTE REPOSITORY GITHUB

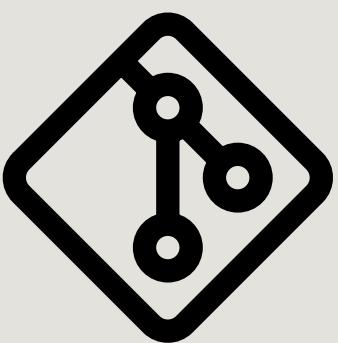


GitHub là một nền tảng lưu trữ đám mây, cung cấp môi trường giúp các lập trình viên có thể tham gia cộng tác, lưu trữ và phát triển mã nguồn.

PHÂN BIỆT GIT VÀ GITHUB

	Git	GitHub
Loại công cụ	Hệ thống quản lý mã nguồn phân tán (DVCS)	Dịch vụ lưu trữ mã nguồn trực tuyến
Nơi chạy	Chạy trên máy cục bộ (local)	Chạy trên nền tảng web (cloud-based)
Chức năng chính	Theo dõi và quản lý thay đổi của mã nguồn	Lưu trữ, chia sẻ, cộng tác và quản lý dự án
Cần internet không?	Không cần	Cần internet để đồng bộ dữ liệu
Hỗ trợ giao diện người dùng?	Chủ yếu dùng qua dòng lệnh (CLI)	Có giao diện đồ họa trên web
Hỗ trợ hệ thống khác?	Có thể dùng với nhiều dịch vụ như GitHub, GitLab, Bitbucket	Chỉ là một nền tảng hỗ trợ Git

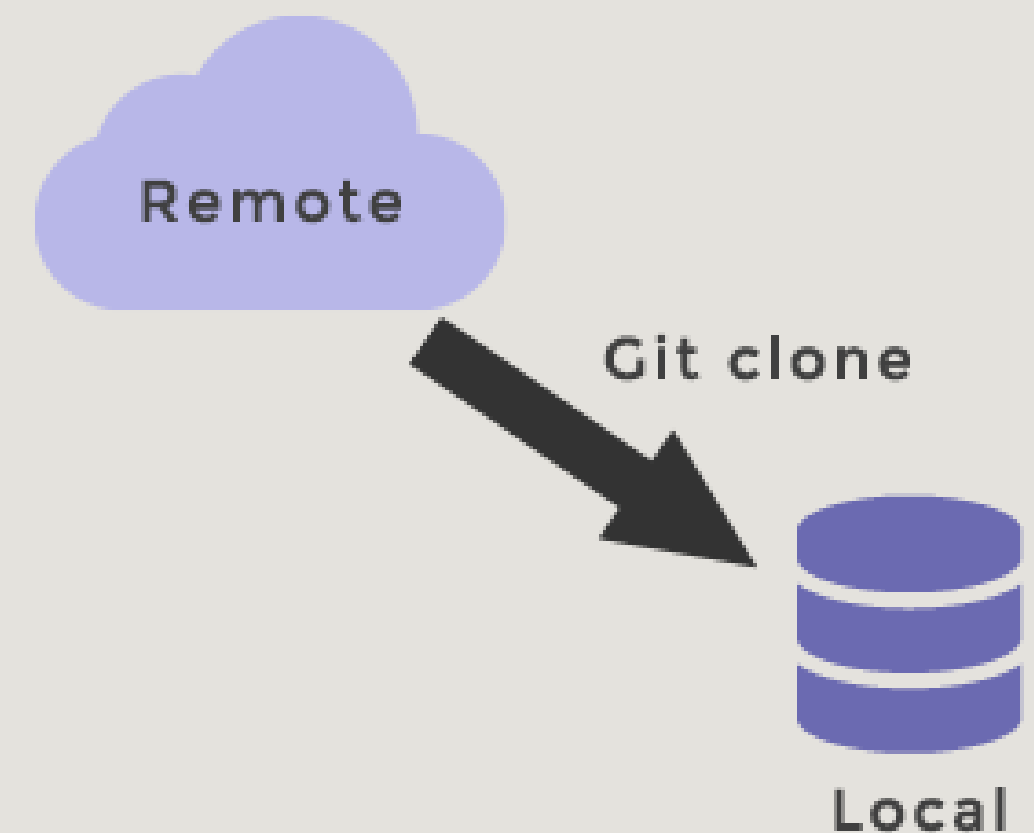
CÁC CÂU LỆNH CƠ BẢN



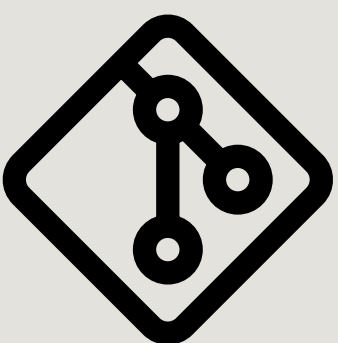
GIT CLONE

Sao chép một Remote repository về môi trường local trong máy tính.

Lệnh: **git clone <url>**



CÁC CÂU LỆNH CƠ BẢN



GIT REMOTE

Xem và quản lý các kết nối giữa các Repository.

Lệnh:

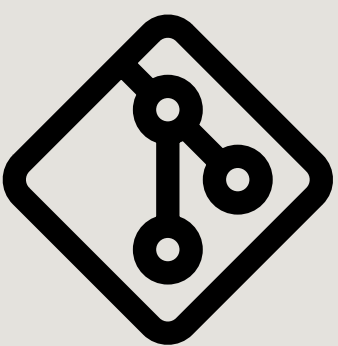
- **git remote**
- **git remote -v**
- **git remote show <tên_remote>**

Thêm, đổi tên, xóa Remote

Lệnh:

- **git remote add <tên_ngắn> <url>**
- **git remote rename <tên_cũ> <tên_mới>**
- **git remote remove <tên>**

CÁC CÂU LỆNH CƠ BẢN



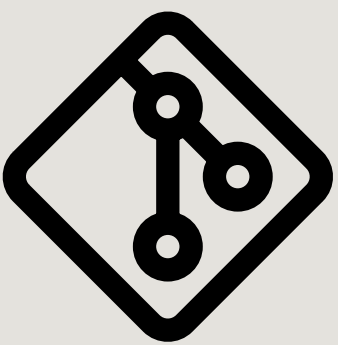
GIT FETCH

Lấy toàn bộ code mới từ máy chủ về máy tính của bạn, NHƯNG KHÔNG tự động gộp (merge) vào code bạn đang viết.

Lệnh:

- **git fetch origin**
- **git fetch --all**
- **git fetch origin main**

CÁC CÂU LỆNH CƠ BẢN



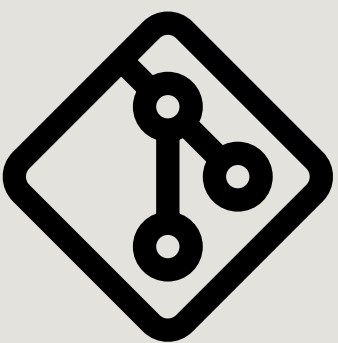
GIT PULL

Sự kết hợp của tải code từ remote (fetch) và GỘP (merge) code đó vào nhánh bạn đang làm việc.

Lệnh:

- **git pull**
- **git pull origin main**

CÁC CÂU LỆNH CƠ BẢN



GIT PUSH

Đẩy các commit mới ở nhánh hiện tại ở local repo lên server (remote repo).

Lệnh:

- **git push <tên_remote> <tên_nhánh>**
 - **git push origin master**

THANK
YOU

